Ex4: Information Retrieval using NLTK

Learning Objective:
The objective of this experiment is to enable students to understand the text preprocessing pipeline and implement a basic information retrieval system using TF-IDF vectorization, Cosine Similarity, NLTK.

Aim: To preprocess customer reviews using NLTK and build a retrieval system using TF-IDF and cosine similarity.

Problem Statement:
An e-commerce company receives thousands of customer reviews for its products every day. Customer support executives often need to retrieve reviews that are similar to a user's search query (e.g., "great product with fast shipping" or "poor quality") to quickly understand customer feedback and address common concerns.

As an NLP Engineer, your task is to preprocess customer review text, convert the reviews into numerical feature vectors using the TF-IDF (Term Frequency–Inverse Document Frequency) technique, and develop a simple information retrieval system that returns the most relevant customer reviews for a given search query using Cosine Similarity.

Tasks to Perform:

1. Use Amazon Fine Food Reviews Dataset from Kaggle and load it into a Pandas DataFrame.
2. Install and import the required Python libraries:
3. Import the required NLTK resources:

Clean the dataset

4. Select the Text column containing customer reviews, Remove missing or null values, Remove duplicate reviews (if any), Retain only the first 10,000 reviews for faster processing.

Text Preprocessing

5. Create a preprocessing function to perform the following operations on each review:
   Convert text to lowercase, Remove punctuation, numbers, and special characters, Tokenize the text into words, Remove English stopwords, Join the cleaned tokens into a single string.
6. Apply the preprocessing function to all reviews and store the cleaned reviews in a new column named Cleaned_Text.
7. Display the original and cleaned versions of any five reviews.
8. Convert the cleaned review text into TF-IDF feature vectors. Display: Shape of the TF-IDF matrix, Total number of unique terms (vocabulary size).

Information Retrieval

10. Implement a document retrieval function that: Accepts a user query, Applies the same preprocessing steps used for the reviews, Converts the query into a TF-IDF vector, Computes Cosine Similarity between the query and all review vectors, Retrieves the Top 5 most similar reviews, Display Similarity score, Original review, Cleaned review.

Query Evaluation

11. Test the retrieval system using the following sample queries: "great product with fast shipping", "disappointed", "excellent taste", "poor packaging"
12. Compare the retrieved reviews for different queries and comment on their relevance.

Solution:

```
# pip install nltk scikit-learn pandas
```

```
import re
import pandas as pd
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nltk.download("punkt")
nltk.download("punkt_tab")
nltk.download("stopwords")

# Load and clean the dataset
df = pd.read_csv("Reviews.csv")
df = df[["Text"]].dropna().drop_duplicates().head(10000).reset_index(drop=True)
print(df.shape)

# Text preprocessing
stop_words = set(stopwords.words("english"))

def preprocess(text):
    text = text.lower()
    text = re.sub(r"[^a-z\s]", " ", text)          # remove punctuation/numbers
    tokens = word_tokenize(text)
    tokens = [w for w in tokens if w not in stop_words]
    return " ".join(tokens)

df["Cleaned_Text"] = df["Text"].apply(preprocess)
for i in range(5):
    print("Original:", df["Text"][i])
    print("Cleaned :", df["Cleaned_Text"][i], "\n")

# TF-IDF
tfidf = TfidfVectorizer()
X = tfidf.fit_transform(df["Cleaned_Text"])
print("TF-IDF matrix shape:", X.shape)
print("Vocabulary size:", len(tfidf.vocabulary_))

# Retrieval
def search(query, top_k=5):
    q_vec = tfidf.transform([preprocess(query)])
    scores = cosine_similarity(q_vec, X).ravel()
    for i in scores.argsort()[::-1][:top_k]:
        print(f"Score   : {scores[i]:.4f}")
        print("Original:", df["Text"][i])
        print("Cleaned :", df["Cleaned_Text"][i], "\n")

queries = ["great product with fast shipping", "disappointed",
           "excellent taste", "poor packaging"]
for q in queries:
    print("=" * 70, "\nQUERY:", q)
    search(q)
```

Observation:

Reviews that contain the words in the query get the highest similarity scores. Queries with more matching words return more relevant reviews. Reviews with similar meaning but different words (for example "bad" for "poor") are not retrieved because TF-IDF matches only the words.

Result:

The reviews were preprocessed, converted to TF-IDF vectors, and the top 5 similar reviews were retrieved for each query.